Bank Product Subscription Prediction Model:

This document focuses on the technical aspects of the project and contains steps and screenshots relevant to the section.

For detailed information, please refer to README.txt and the 12510050_MLOPS.ipynb files.

1. Data Handling and EDA

- Use of one hot encoding for categorical variables
- StandardScaler used for scaling numerical features so that higher values do not dominate the model.
- The column “duration” removed.
- Data and related features plotted for visualization and understanding the impact.

```python
# Identify categorical columns (dtype as object)
categoricalCols = df.select_dtypes(include=['object']).columns.tolist()

# Exclude the target variable 'y' from encoding
categoricalCols = [col for col in categoricalCols if col != 'y']

# One-Hot Encoding using get_dummies
df_encoded = pd.get_dummies(df, columns=categoricalCols, drop_first=True)

# Preview encoded data
df_encoded.head()
```

[0 1]

	age	campaign	pDays	previous	empVarRate	consPriceIdx	consConfIdx	eurIbor3M	nrEmployed	y
0	30	2	-1	0	-1.8	92.893	-46.2	1.313	5099.1	0
1	39	4	-1	0	1.1	93.994	-36.4	4.855	5191.0	0
2	25	1	-1	0	1.4	94.465	-41.8	4.962	5228.1	0
3	38	3	-1	0	1.4	94.465	-41.8	4.959	5228.1	0
4	47	1	-1	0	-0.1	93.200	-42.0	4.191	5195.8	0

5 rows × 54 columns

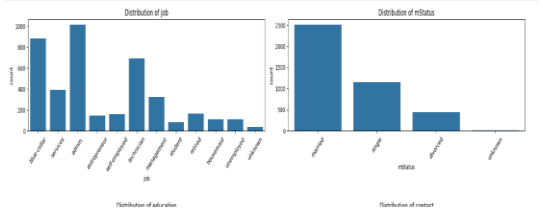
```python
# Plot to visualize the important categorical variables

categoricalCols = [
    'job', 'mStatus', 'education', 'contact',
    'default', 'housing', 'loan', 'poutcome'
]

plt.figure(figsize=(18, 30))

for i, col in enumerate(categoricalCols, 1):
    plt.subplot(4, 2, i)
    sns.countplot(x=col, data=df)
    plt.title(f"Distribution of {col}")
    plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



Distribution of education

Distribution of poutcome

2. Training the model and preparation for further deployment

- 80% Train and 20% Test split of data.
- Used Logistic Regression for building the model.
- Evaluated the model with relevant metrics like recall, F1 Score, Confusion Matrix.

```python
# Split the dataset into training and testing sets (80% Train - 20% Test)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Shapes
print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

```
X_train: (3295, 53)
X_test: (824, 53)
y_train: (3295,)
y_test: (824,)
```

1.4 Model Evaluation

```python
# Predictions on test data
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Appropriate metrics for evaluation -- precision, recall, f1-score
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# Confusion matrix
print("Confusion Matrix:\n")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.8325242718446602

Classification Report:

              precision    recall  f1-score   support

           0       0.94      0.86      0.90       734
           1       0.34      0.58      0.43        90

    accuracy                           0.83       824
   macro avg       0.64      0.72      0.67       824
weighted avg       0.88      0.83      0.85       824

Confusion Matrix:

[[634 100]
 [ 38  52]]
```

Trained Model: 12510050_logistic_model.pkl

Scaler: 12510050_scaler.pkl

3. <u>Docker Containerization</u>

- Created Dockerfile
- Created app.py
- Installation dependencies mentioned in requirements.txt
- Packaged Flask, model and scaler to a Docker image
- Built docker
- Container run locally to test and evaluate

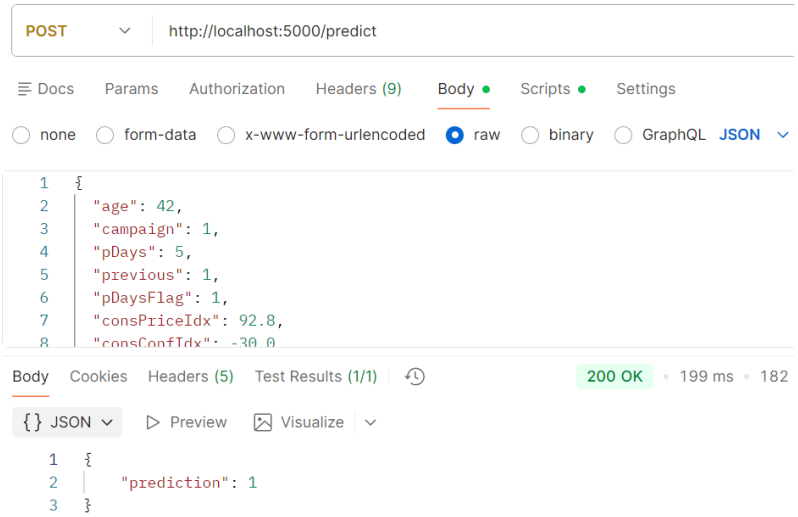
Commands:

docker build -t 12510050-mlops-model .

docker run -p 5000:5000 12510050-mlops-model

```
C:\Users\LENOVO\Downloads\20251126\AMPBA\AMPBA_MLOPS\12510050_mlops_model>docker run -p 5000:5000 12510050-mlops-model
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
```

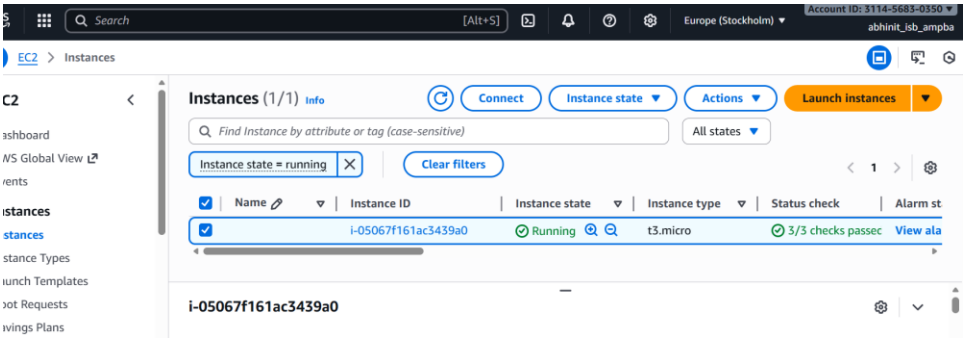
Test (Postman):



4. <u>AWS Deployment</u>

- Launched an EC2 instance with AMI and t3.micro
- Configured inbound rules to accept port 5000.
- Installed docker on EC2.
- Copied project files to EC2 using SCP.
- Built the docker image on the instance.
- Ran the docker container on EC2.
- Ran the results on the public IP.

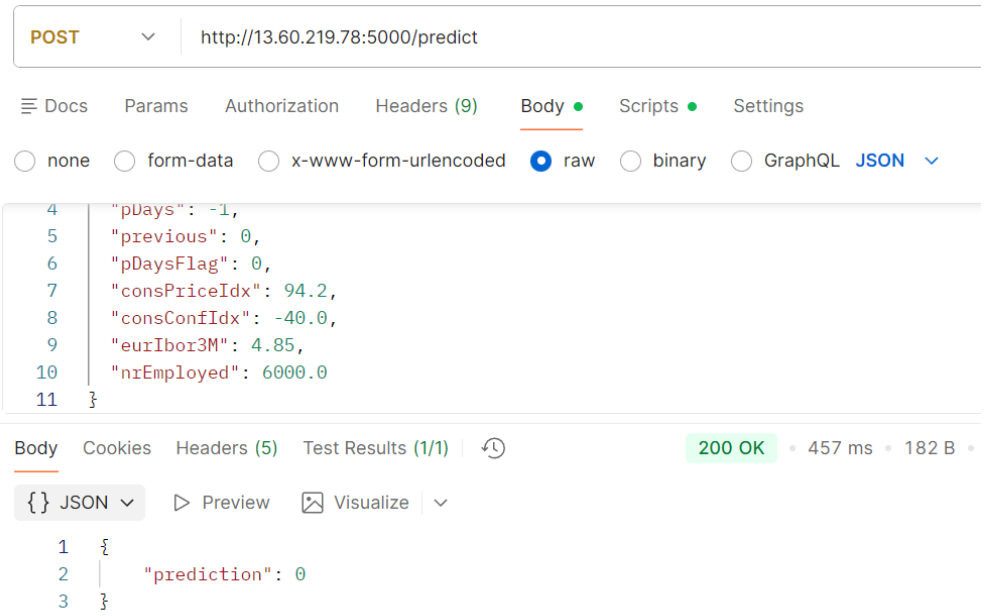
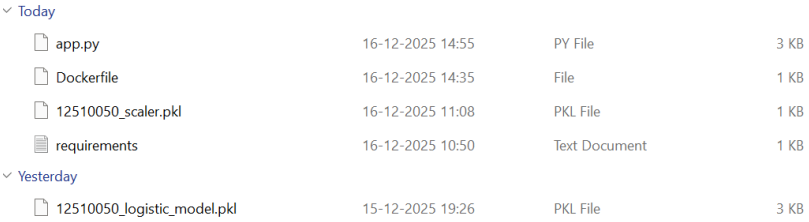
AWS Instance:



EC2 Docker:

```
[ec2-user@ip-172-31-17-12 12510050_mlops_model]$ docker images
REPOSITORY              TAG       IMAGE ID       CREATED          SIZE
12510050-mlop-model     latest    17b98f1df66c   7 minutes ago    460MB
12510050-mlops-model    latest    17b98f1df66c   7 minutes ago    460MB
[ec2-user@ip-172-31-17-12 12510050_mlops_model]$ docker run -p 5000:5000 12510050-mlops-model
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a product
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
122.171.22.2 - - [16/Dec/2025 13:44:05] "POST /predict HTTP/1.1" 200 -
122.171.22.2 - - [16/Dec/2025 13:44:21] "POST /predict HTTP/1.1" 200 -
```

Running the prediction using public IP:



Required Folder Structure and files: